

Reviewer Pack — Minimal Rank of Hodge Structures in Algebraic Geometry vs Mo...

Entry 89e40a48ded3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:41:50 UTC

Minimal Rank of Hodge Structures in Algebraic Geometry vs Monotone Circuit Tautology Degree

Entry ID: 89e40a48ded3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 04:41:50 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

{‘quantitative’: ‘For any n -vertex graph G , the minimal rank of its associated Hodge structure is upper-bounded by the tautology degree of a monotone circuit computing the OR of all vertices in G .’, ‘falsifier-friendly’: ‘If there exists an n -vertex graph G for which the minimal rank of its Hodge structure is greater than the tautology degree of a monotone circuit computing the OR of all vertices, then the conjecture is falsified.’}

Rationale (proposer’s reasoning):

{‘expose_structure’: ‘Hodge structures encode deep geometric information about varieties. If this conjecture holds, it would imply a strong link between algebraic geometry and circuit complexity, revealing potentially new insights into both fields.’, ‘natural_group_action’: ‘The group action of the general linear group on Hodge structures provides a natural way to study invariants under group actions, which is essential for defining the invariant $\psi(C)$ on monotone circuits.’}

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98b1e7ade225243c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an n -vertex graph G , if the minimal rank of its Hodge structure is greater than or equal to the tautology degree of a monotone circuit computing the OR of all vertices (metric_mean $\geq$ tautology_degree), then the conjecture is supported. If any seed produces a metric_mean $<$ tautology_degree, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge theory" AND "monotone circuit complexity" - "tautology degree" IN "algebraic geometry" - "minimal rank Hodge structure" AND OR("monotone circuit" "complexity theory" "circuit complexity")

Top relevant hits considered: - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/math/0502387v3] Multiplier ideals in algebraic geometry - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        G = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    G[i][j] = G[j][i] = 1
        return G

    def tautology_degree(G):
        n = len(G)
        max_degree = 0

        for s in range(2**n):
            degree = 0
            for j in range(n):
                if (s >> j) & 1:
```

```

        for k in range(j + 1, n):
            if G[j][k] and (s >> k) & 1:
                degree += 1
        max_degree = max(max_degree, degree)

    return max_degree

def minimal_rank(G):
    n = len(G)
    I = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]

    # Gaussian elimination to find the rank
    for i in range(n):
        if G[i][i] == 0:
            found = False
            for k in range(i + 1, n):
                if G[k][i] != 0:
                    G[i], G[k] = G[k], G[i]
                    I[i], I[k] = I[k], I[i]
                    found = True
            break
        if not found:
            return i

    for j in range(n):
        if j == i:
            continue
        factor = -G[j][i] / G[i][i]
        for k in range(n):
            G[j][k] += factor * G[i][k]
            I[j][k] += factor * I[i][k]

    return n

n = random.choice([5, 10, 15, 20, 30, 40])
G = generate_graph(n)

rank = minimal_rank(G)
degree = tautology_degree(G)

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= degree,
    "counterexample": "" if rank <= degree else f"Graph with n={n}, rank={rank}, degree={degree}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

metric_values = [r["metric_value"] for r in results]
conjecture_holds = all(r["conjecture_holds"] for r in results)

if conjecture_holds:
    mean = sum(metric_values) / len(metric_values)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = len(results) / len(seeds)
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot determine if the conjecture is supported or falsified based on the pre-registered support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12045
2	preregistration	ollama_remote	glm4:latest	0	0	5517
3	novelty	ollama_remote	glm4:latest	0	0	4737
4	novelty	ollama_remote	glm4:latest	0	0	6684
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10843
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9169
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12113
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10085
9	judge	ollama_remote	glm4:latest	0	0	10547

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 81739 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/89e40a48ded3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/89e40a48ded3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/89e40a48ded3.tar.gz (if generated)